# Increasing the Dependability of Internet-of-Things Systems in the context of End-User Development Environments

João Pedro Dias

jpmdias@fe.up.pt

Supervision by:
Hugo Sereno Ferreira, PhD
João Pascoal Faria, PhD

In partial fulfillment of requirements for the degree of *Doctor of Philosophy in Informatics Engineering* by the Doctoral Program in Informatics Engineering (ProDEI)

April 1, 2022 — Porto, Portugal

U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

# Table of Contents



1 **Introduction**
- context
- motivation
- research statement

2 **Fundamentals**
- background & state-of-the-art
- automation survey

3 **Patterns**
- pattern language
- support
- error detection
- recovery and maintenance

4 **Dependable and Autonomic**
- serverless dynamic allocation
- visual dynamic orchestration
- self-healing

5 **End-User Development**
- visual real-time feedback
- conversational assistants

6 **Conclusion**
- research goals revisited
- outcomes and contributions
- future work

# 1 Introduction

- **Internet-of-Things** (IoT), i.e., the interconnection of everyday things over the Internet, enabled the **automation of everyday tasks at large** by fusing the physical and virtual realms, supported by **networked devices with sensing and actuating capabilities**.
- IoT usage **across application domains** (*e.g.*, homes, cities, electrical grids, transportation, cities) makes it **infeasible to depend on individuals with specific expertise** to configure and operate all of these systems, as there is already a shortage of individuals with such technical expertise.
- As the reliance on these systems increases their **dependability becomes a core issue**, as their misbehaviour can lead to undesirable side-effects.

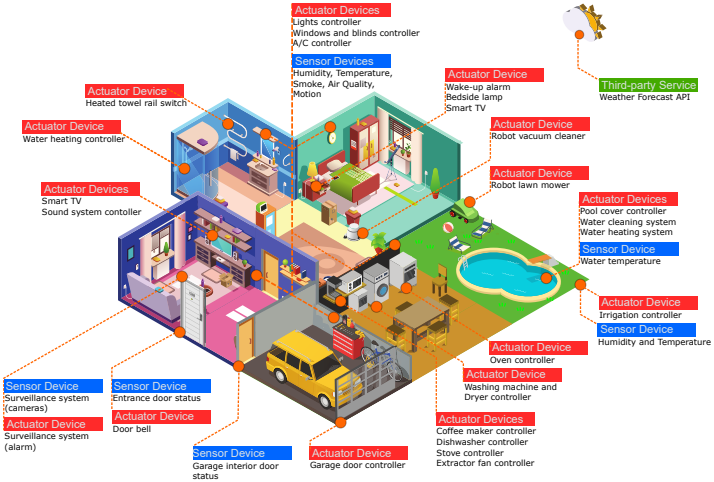



Figure 1: Smart home motivational scenario.

- IoT results from the **combination of knowledge from different fields** of hardware and software research, making **IoT systems complex in a mostly-unique fashion and at a large-scale**.
- The **market-driven rapid development of IoT** solutions by several vendors **without a consensus on standards, guidelines, or best practices** has lead to an ever-growing **technological fragmentation** and a **generalized dependency on vendor-locked centralized cloud** infrastructure.
- This fast growth is happening with a **generalized disregard** by practitioners **for the correct operation of these systems when errors occur**, i.e., their dependability. This fact also **hinders the ability of the users to make their systems more dependable**.

RQ1 What are the **unique characteristics** of IoT systems that **make them complex**, and how does such complexity **impact the end-user** ability to configure their **dependable systems**?

RQ2 Are there **recurrent problems** concerning the lifecycle of IoT systems, and what are the **prevalent solutions** that address them?

RQ3 What can be **improved** concerning the IoT systems' **dependability**?

RQ4 How can the **mechanisms identified** in RQ2 be **leveraged by the end-users** of IoT systems?

RQ5 How can the **end-user's ability to manage the IoT systems'** lifecycle be **improved without requiring specific expertise** nor **hindering the systems' dependability**?

> ***H:*** *It is possible to enrich* ***IoT-focused end-user development environments*** *in such a way that the resulting systems have a* ***higher dependability degree****, with the* ***lowest impact on the know-how*** *of the (end-)users.*

Using **Node-RED** as a reference *development environment*, the goal is to:

(a) provide the building-blocks that allow user's to address dependability concerns;
(b) enable the resulting systems to *self*-address some errors of their parts with minimal disruption;
(c) not increase the complexity of achieving systems that perform as the (end-)user requires.

# 2 Fundamentals

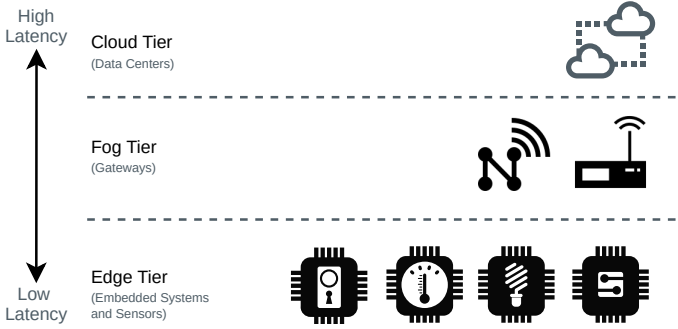



Figure 2: Three-tier IoT view.

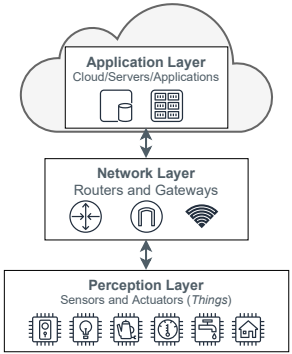



Figure 3: Three-layer IoT view.

▷J. P. Dias, F. Couto, A. C. R. Paiva, and H. S. Ferreira. A brief overview of existing tools for testing the internet-of-things.
In *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pages 104–109, Apr. 2018
▷ J. P. Dias, J. P. Faria, and H. S. Ferreira. A reactive and model-based approach for developing internet-of-things systems.
In *2018 11th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 276–281, Sept. 2018
▷ J. P. Dias, A. Restivo, and H. S. Ferreira. Designing and constructing internet-of-things systems: An overview of the ecosystem.
*(Submitted to) Internet of Things: Engineering Cyber Physical Human Systems Journal*, 2022

### Heterogeneity

Protocols, computing capabilities, and standards.

### Highly-distributed

Geographical and logical.

### Virtual & Physical realms

Increased risks from failure side-effects.

### Interoperability

Vendor-lock and lack of consensus on standards.

### Real-time & QoS

Trigger-action rules and user commands.

### Application Domains

Industry, environment, and society.

### Large-scale

Devices, services, and applications.

### End-user focus

Low-code development and multi-modal interaction.

### Privacy and Security

Data ownership, weak encryption, and misconfigurations.

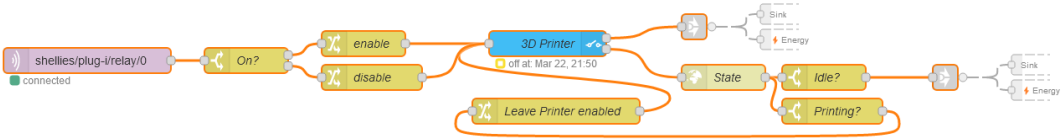



Figure 4: Node-RED example flow.

- Traditional programming approaches **have limitations when coping with the IoT system's complexity** and **demand considerable levels of technical expertise** from their users.
- Approaches that leverage **abstractions**, *e.g.*, **drag-n-drop visual notations** or **voice interactions**, became the go-to development solutions, and commonly use **models and/or mashup** techniques.
- These low-code solutions commonly **limit the complexity of the logic flows being created** or **heavily depend on the user expertise to create flows that behave as they intend to**.
- Most of these solutions also have **little to no debug capabilities**, and **lack of verification and validation mechanisms**.

Node-RED is an open-source (≈14300 stars on GitHub) **low-code visual programming solution** that has a primary focus on ***event-driven* IoT development**, and follows a **centralized architecture**.

Node-RED has, however, several issues and limitations:

1. all the **computation is performed in one instance** (limiting computational distribution);
2. computational heavy **tasks will impact the performance of the whole system**, and faults in a single flow can lead to service disruption;
3. there is **no isolation of execution contexts** which can raise both security and privacy issues.
4. the web-based development **interface is highly-coupled with the runtime**;
5. **no mechanisms to verify the structural correctness of the developed flows** (*e.g.*, types).

FogFlow, DDFlow, and uFlow are some of the **solutions that have been purposed to attain the distribution of computation in Node-RED** by **decomposition of flows and allocation of tasks across available computational resources**.

▷ M. Silva, J. P. Dias, A. Restivo, and H. S. Ferreira. A review on visual programming for distributed computation in iot.
In *Proceedings of the 21st International Conference on Computational Science (ICCS)*. Springer, 2021

"(...) **dependability** of a system is the ability to avoid service failures that are more frequent and more severe than is acceptable." (Avizienis *et al*. , 2004)

Dependability encompasses the following attributes:

| | |
|---|---|
| Availability | readiness for correct service; |
| Reliability | continuity of correct service; |
| Safety | absence of catastrophic consequences on the user(s) and the environment; |
| Integrity | absence of improper system alterations; |
| Maintainability | ability to undergo modifications and repairs. |

Attributes attained by using **fault prevention**, **fault tolerance**, **fault removal**, and **fault forecasting**.

IBM Research proposed **autonomic computing** as a way of coping with the continuous growth in the **complexity of operating, managing, and integrating computing systems**. An autonomic computing systems needs to *know and understand itself*, thus must be:

- Automatic capable of **controlling its own operations** without any manual external intervention;
- Adaptive able to **adapt its operation to cope with runtime changes** in its operational environment;
- Aware able to **monitor operating conditions to assert if its operation meets the service goals**.

| Self-Configuration | Self-Heal | Self-Optimization | Self-Protection |
|---|---|---|---|
| Ability to readjust on-the-fly to cope with dynamically changing environments. | Ability to **automatically discover, diagnose, and react to, or recover from, failures**. | Optimize resource utilization to improve the quality of the service over time. | Anticipating, detecting, identifying, and protecting itself from attacks. |

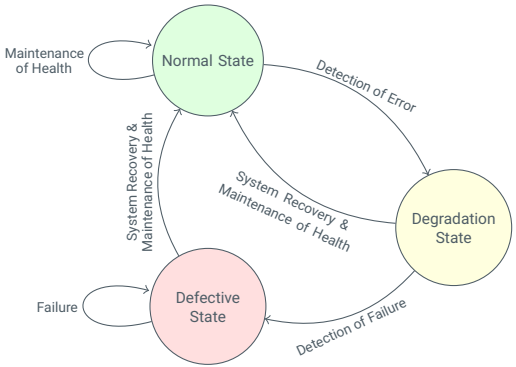



Figure 5: State transactions of a self-healing system.

- Most **IoT systems are open-loop** — there is **no direct feedback-loop** from the sensing part to the acting part, thus hindering the adoption of resilience improvement mechanisms.
- Most fault-tolerance mechanisms follow a **reactive** behaviour, using strategies such as system watchdogs and supervisors.
- There are only a **few works that propose the use of autonomic computing in IoT** systems, and even fewer that purpose the use of self-healing.
- Some authors propose the use of runtime verification to enable a system to self-heal, however they typically **depend on a formal specification of the system to properly work**.

## Survey

- An online survey was distributed among 20 participants which only requirement was to fill in a text box with as many **automation ideas** as they could think of;
- Participants were **provided a 3D model of a common house and a list of IoT devices** as inspiration and common baseline;
- The survey resulted in a total of **177 automation scenarios**;
- The results were categorized (11 categories) in accordance with the (1) **sensors involved**, (2) **type of actuator**, and (3) **periodicity**.

## Observations

- ≈**94.3% fitted into one of the 11 defined categories**;
- The scenarios differ in terms of the **granularity of application**, **complexity** (*e.g.*, number of devices), and **writing style** (with most being close to conditional logic).
- Most scenarios are expressed in the format of "when *condition*, then *action*", or "*action*, when *condition*", also known as **Trigger-action programming (TAP)**.
- ≈**29% mentioned Boolean operators**;
- ≈**7% contained chained operations**;
- ≈**27%** are too generic, **depending on contextual awareness and user preferences**.

# 3 Patterns

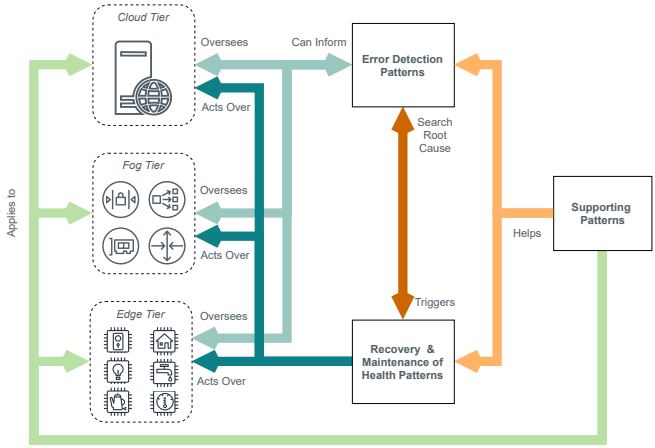
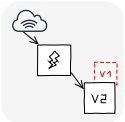



Figure 6: Pattern language map.

## Contributions

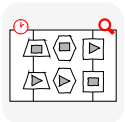
- A compendium of **34 patterns** patlets describing problem-solution pairs in the IoT systems context.
- Focused on **fault-tolerance while leveraging autonomic computing strategies**.
- Each pattern has, at least, **three independent examples of use as reported by the literature/industry**.
- Most **patterns can be used at different tiers of the IoT system**, depending on the concrete implementation being used.



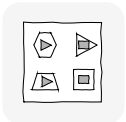

Middleman Update



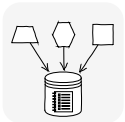
Testbed



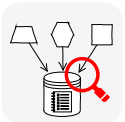
Simulation-based Testing



Device Registry



Device Raw Data Collector



Device Error Data Supervisor



Predictive Device Monitor

Figure 7: Supporting patterns.

▷ A. Ramadas, G. Domingues, J. P. Dias, A. Aguiar, and H. S. Ferreira. Patterns for Things that Fail.
In *Proceedings of the 24th Conference on Pattern Languages of Programs*, PLoP '17. ACM, 2017

▷ J. P. Dias, H. S. Ferreira, and T. B. Sousa. Testing and deployment patterns for the internet-of-things.
In *Proceedings of the 24th European Conference on Pattern Languages of Programs*, EuroPLop '19. ACM, 2019

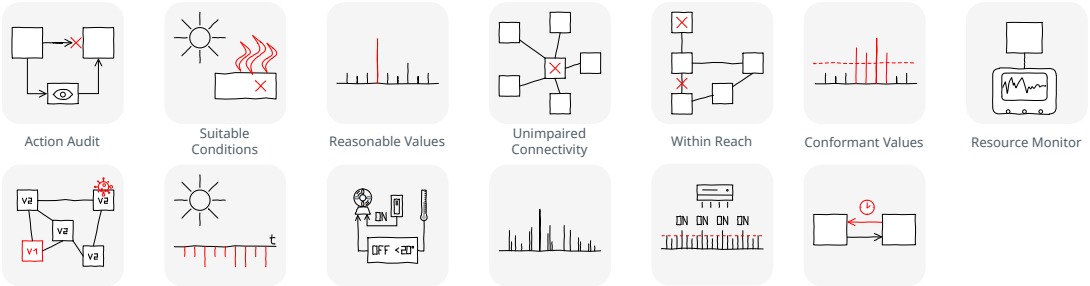



Figure 8: Error detection (probes) patterns.

▷ J. P. Dias, T. B. Sousa, A. Restivo, and H. S. Ferreira. A pattern-language for self-healing internet-of-things systems. In *Proceedings of the 25th European Conference on Pattern Languages of Programs*, EuroPLop '20. ACM, 2020

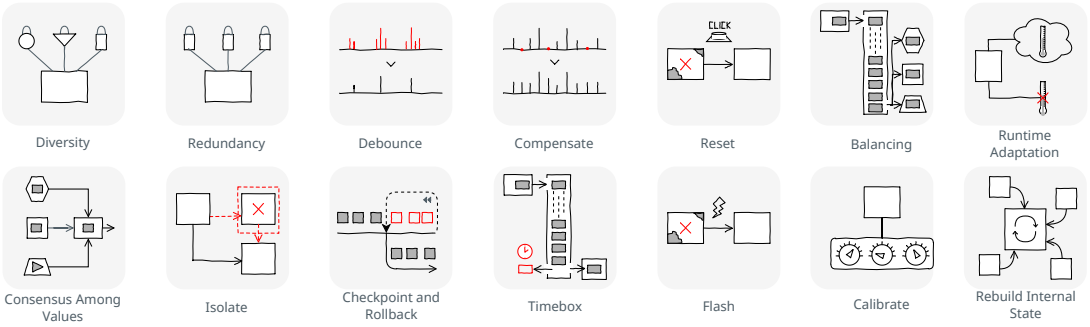



Figure 9: Recovery and maintenance of health patterns.

▷ J. P. Dias, T. B. Sousa, A. Restivo, and H. S. Ferreira. A pattern-language for self-healing internet-of-things systems. In *Proceedings of the 25th European Conference on Pattern Languages of Programs*, EuroPLop '20. ACM, 2020

# 4 Dependable and Autonomic

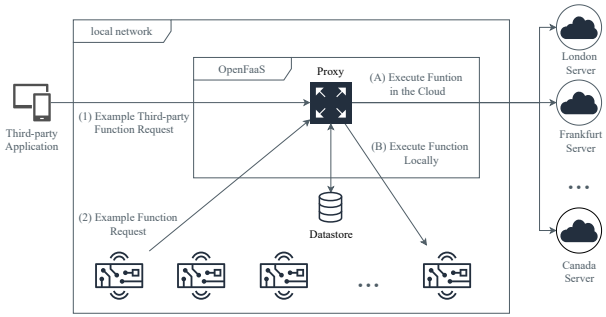



Figure 10: High-level overview of the solution operation.

## Outcomes

- One of the first works in the literature that leverages the concept of serverless in IoT domain.
- Ability to **dynamically allocate functions**, i.e, computational tasks, taking into account **runtime constraints, pre-conditions, and device's features**.
- **Exploration *versus* exploitation to continuously improve system performance**, i.e., response time.

▷ D. Pinto, J. P. Dias, and H. Sereno Ferreira. Dynamic allocation of serverless functions in iot environments. In *2018 IEEE 16th International Conference on Embedded and Ubiquitous Computing (EUC)*, pages 1–8, Oct. 2018

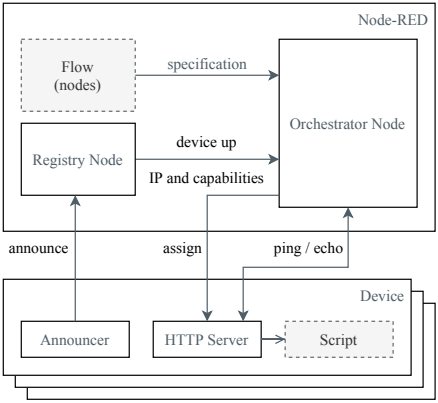



Figure 11: Proof-of-concept overview.

## Details

- Node-RED was used to define programs (as flows) and modified to allow **send tasks to other devices in the network**;
- Two *nodes* were added to Node-RED: *Registry*, which **maintains a list of available devices and their capabilities**, and the *Orchestrator*, which **partitions *flows* and assigns tasks to the devices**;
- Each **device runs a customized MicroPython firmware** to ease the task allocation process;
- Each *allocatable* node has two implementations, one Node-RED compatible and another compatible with the device's firmware.

▷ M. Silva, J. P. Dias, A. Restivo, and H. S. Ferreira. Visually-defined real-time orchestration of iot systems.
In *Proceedings of the 17th International Conference on Mobile and Ubiquitous Systems*, MOBIQUITOUS 2020. ACM, 2020

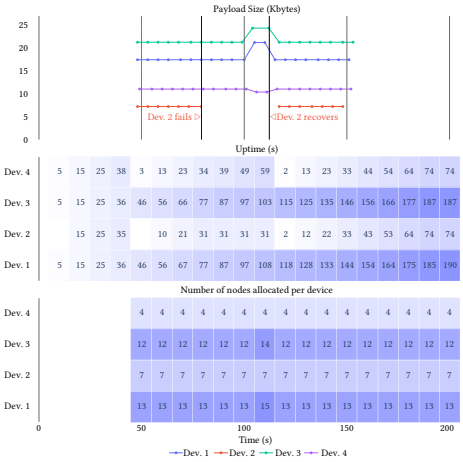



Figure 12: Out-of-Memory experiment.

## Experiments & Results

Two experimental setups were drafted and a total of 13 experimental scenarios were run per setup, using a mix of simulated devices and real devices. We observed **improvements** in terms of:

- **Resilience** as the system **handles device failures and memory constraints** dynamically;
- **Elasticity**, with an IoT system with up to **50 devices running in a decentralized fashion with devices being added and removed in runtime**;
- **Efficiency** given that most of the overheads come from the **extra latency introduced by the communication channel**.

**Known limitations**: ability to find a suitable configuration to orchestrate all the tasks given constraints in computing power and poor use of historical data usage for deciding task allocation.




Figure 13: Node-RED self-healing nodes.

## Details

- Node-RED *nodes* that correspond to **one or more self-healing patterns**, allowing to **detect and mitigate/recovery from IoT system errors and failures** within Node-RED flows.
- Some *nodes* leverage *meta-facilities* that allow **changing a system's behavior during runtime** (*e.g.*, activate/deactivate flows).
- The feasibility of using the *nodes* to mitigate some types of failures **was tested under 6 experimental scenarios on a physically deployed testbed** (*SmartLab*).
- Some patterns that do not have a direct representation as Node-RED *nodes* since they depend on specific capabilities of devices.

The nodes are publicly available and can be installed (>1500 downloads): `https://flows.nodered.org/node/node-red-contrib-self-healing`.

▷ J. P. Dias, B. Lima, J. P. Faria, A. Restivo, and H. S. Ferreira. Visual self-healing modelling for reliable internet-of-things systems. In *Proceedings of the 20th International Conference on Computational Science (ICCS)*, pages 27–36. Springer, 2020
▷ J. P. Dias, A. Restivo, and H. S. Ferreira. Empowering visual internet-of-things mashups with self-healing capabilities. In *2021 IEEE/ACM 3rd International Workshop on Software Engineering Research Practices for the Internet of Things (SERP4IoT)*, 2021

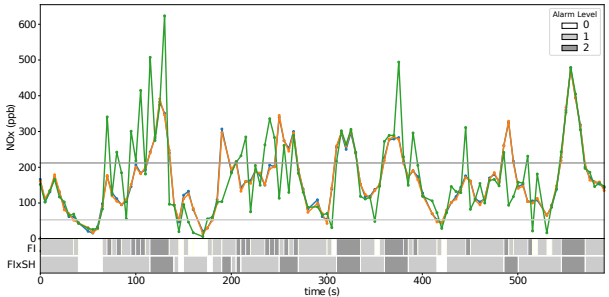



Figure 14: FI experiment to create spikes in the Sensor 3 readings (in green). FI experiment has an overlap of 76.3% to baseline, while FIxSH has an overlap of 97.4%.

## Fault-injection Experiments

Two experimental scenarios (6 experiments) were carried to assess the functioning self-healing mechanisms when faults are injected. The main observations were that:

- The self-healing *nodes* **do not make the system deviate substantially in behavior from the baseline** system;
- The **faults injected are consequential** since there is a deviation on the baseline system in comparison to when no fault is being injected;
- When the faults injected are consequential, the **self-healing system was able to recover from them**, conforming with the normal service.

▷ M. Duarte, J. P. Dias, H. S. Ferreira, and A. Restivo. Evaluation of iot self-healing mechanisms using fault-injection in message brokers. In *2022 IEEE/ACM 4th International Workshop on Software Engineering Research Practices for the Internet of Things (SERP4IoT)*, 2022

# 5 End-User Development

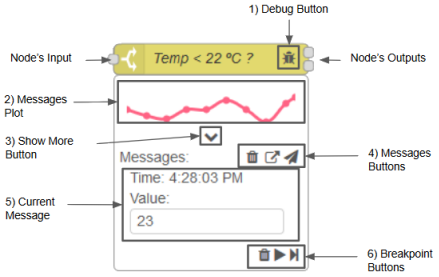



Figure 15: Annotated enhanced *node* visual notation.

## Outcomes

- Modifications to Node-RED development environment to **improve feedback during development** and the **debugging capabilities**;
- An experiment was carried with **20 participants** where they had to complete **2 control tasks and 3 experimental tasks, *i.e.*, debugging, improvement, and implementation** in the original Node-RED or in the modified version;
- The added **enhancements improve the overall development process**, with a **significant reduction of the number of failed attempts to deploy** the systems without fulfilling its requirements;
- The **overall system development time was lower** than with the original Node-RED.

▷ D. Torres, J. P. Dias, A. Restivo, and H. S. Ferreira. Real-time feedback in node-red for iot development: An empirical study.
In *2020 IEEE/ACM 24th International Symposium on Distributed Simulation and Real Time Applications (DS-RT)*, pages 1–8, 2020

| | Jarvis | Google Assistant | Node-RED |
|---|---|---|---|
| One-time action | ● | ● | ● |
| One-time action w/unclear device | ● | · | · |
| Delayed action | ● | · | ● |
| Period action | ● | · | ● |
| Daily repeating action | ● | · | ● |
| Daily repeating period action | ● | · | ● |
| Cancel last command | ● | · | · |
| Event rule | ● | · | · |
| Rules defined for device | ● | · | · |
| Causality query | ● | · | · |

Table 1: Scenario support by different solutions.

## Outcomes

- Jarvis is an alternative approach for **managing IoT spaces in a conversational way**;
- **Casuality queries enable users to understand why something happened**;
- A feasibility experiment was run with **17 participants, which had to complete a total of 5 tasks**;
- The **completion rate of all task has always higher than 85%**, providing evidence that the system might be intuitive enough to be used without previous instruction or formation.
- In terms of **subjective perception, participants pointed conversational assistants as a preferred approach** when compared to visual notations.

▷ J. P. Dias, A. Lago, and H. S. Ferreira. Conversational interface for managing non-trivial internet-of-things systems. In *Proceedings of the 20th International Conference on Computational Science (ICCS)*, pages 27–36. Springer, 2020

▷ Lago, J. Dias, and H. Ferreira. Managing non-trivial internet-of-things systems with conversational assistants: A prototype and a feasibility experiment. *Journal of Computational Science*, 51:101324, 2021

# 6 Conclusion

RQ1 What are the **unique characteristics** of IoT systems that **make them complex**, and how does such complexity **impact the end-user** ability to configure their **dependable systems**?

Essential complexity in IoT comes from the nature of these systems, *i.e.*, their **large-scale**, **heterogeneity**, **highly-dynamic networks**, **end-user-centrism**, and **real-world blending**.

Accidental complexity comes mostly from **time-to-market** forces that makes vendors **disregard best practices or standards**, *e.g.*, cloud-only architectures.

Most **end-user development environments are hindered by this complexity**, where **users are limited in what they can program**, and **are arduous to use/understand as the complexity of the system increases**.

Making IoT systems dependable appears as an even more significant barrier, given that **most development environments do not provide the means to detect errors/failures and configure fallback or recovery strategies**.

RQ2 Are there **recurrent problems** concerning the lifecycle of IoT systems, and what are the **prevalent solutions** that address them?

There are recurrent problems in IoT systems which solutions can be defined and implemented in software, but faults can originate either in software or hardware components.

We have identified a total of 34 problem-solution pairs, *i.e.*, patterns: **seven are considered supporting patterns**, **13 focus on error detection**, and **14** detail solutions to common situations on IoT system operation that either **require the system to recover or, at least, to act to maintain its health**.

**The combination of error detection and recovery patterns allows the system to behave autonomically, *i.e.*, self-heal.**

RQ3 What can be **improved** concerning the IoT systems' **dependability**?

While several fault-tolerance strategies have already been adopted in the IoT domain by researchers and practitioners, the **adoption of mechanisms to distribute system load and avoid single-point-of-failure in IoT scope** is only exploratory and with several pending issues.

Adopting **mechanisms that dynamically allocate computational tasks while adapting to runtime constraints** in IoT allows the system to adapt and operate nominally even when facing disruptions.

The introduction of the notion of **orchestrator on Node-RED** enables users to program their visual flows while allowing the **decentralization of computing, as the computation of nodes of a given flow can happen in any available computational resources**.

RQ4 How can the **mechanisms identified** in RQ2 be **leveraged by the end-users** of IoT systems?

Allowing an end-user to use the discussed patterns **implies that the solution they are using has the built-in mechanisms to support one or more strategies presented** and **leverages the same category of abstraction that the development solution already uses**.

We **have implemented 17 Node-RED nodes**, corresponding to **one or more strategies detailed as possible solutions on 19 different patterns**, that allow the definition of self-healing behaviours.

Node-RED was enhanced with runtime adaptation capabilities by reducing the always-on dependency by **allowing to allocate computing tasks among available resources during runtime in a visual and transparent fashion**.

We **evaluated these contributions both in simulated and physical testbeds using scenario-based experiments and fault-injection**, showcasing their **feasibility and improvements when compared to the baseline**.

RQ5 How can the **end-user's ability to manage the IoT systems'** lifecycle be **improved without requiring specific expertise** nor **hindering the systems' dependability**?

To **overcome the limitations in terms of runtime feedback to the end-user and ease of understanding the configured system** at any given point in time, **we enriched the Node-RED visual abstractions used to improve the inspection of the system**, with **significant improvements in the user's capability of understanding the system and reduction in development time**.

Additionally, to **improve the user's ability to understand the configured automation's at a given time we adopted voice assistants**, showcasing the **feasibility of using such assistants to query the system** and, in some cases, **understand the causality between events**.

## It is possible to enrich IoT-focused end-user development environments...

As IoT systems are mostly used by non-technical users, we selected the **Node-RED visual development solution** as a reference solution in our research.

## ...in such a way that the resulting systems have a higher dependability degree...

By identifying recurrent problems of IoT systems we **identified a set of patterns that can be used to improve the dependability of IoT systems**, patterns that can be used in tandem to make the **system self-heal**. We also asserted the feasibility of using Node-RED as a **visual orchestrator of the system**, **allowing end-users to leverage the computational resources available**, responding autonomically to runtime changes, while **reducing the dependency on Node-RED itself**.

## ...with the lowest impact on the know-how of the (end-)users.

We successfully implemented a subset of the patterns as **extensions to Node-RED that allow users to configure self-healing behaviors**, thus enabling them to enhance their systems' dependability **without necessarily increasing the complexity of the development environment**. The contributions on **descentralizing Node-RED computation are also transparent to the end-user**. The use of **voice assistants as a supporting tool to visual approaches** can be used to improve the user understanding about in-place automations and the causality of certain events.

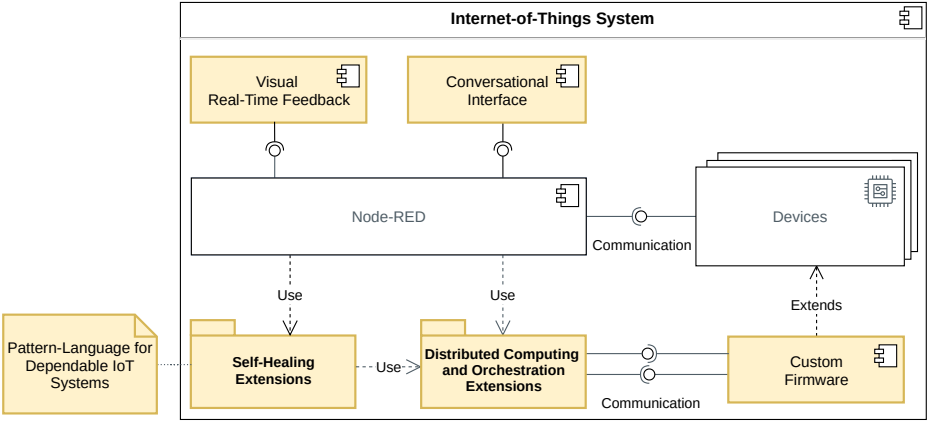



Figure 16: High-level overview of the main contributions of this work.

- Study the **adoption and relevance of the identified patterns** in the community by distributing a survey among IoT practitioners and developers;
- **Improve and mitigate the known limitations** regarding the **dynamic distribution and orchestration of computing tasks in IoT** systems;
- Focus on developing the firmware that runs on the edge devices, **exploring solutions such as the use of WASM and RTOS**;
- **Expand the Node-RED self-healing extension by implementing more *nodes*** corresponding to the remaining identified patterns;
- Address **other aspects of autonomic computing beyond self-healing**;
- **Further research on improving the IoT development environments**, specially the ones that focus end-users with little to no experience or technical knowledge, *e.g.*, by combining visual programming and voice assistants.

# References I


- W. Torres-Pomales. Software Fault Tolerance: A Tutorial.
  *NASA / TM-200-210616*, 2000
- A. G. Ganek and T. A. Corbi. The dawning of the autonomic computing era.
  *IBM systems Journal*, 42(1):5–18, 2003
- A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr. Basic concepts and taxonomy of dependable and secure computing.
  *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004
- J. Spolsky. The law of leaky abstractions.
  In *Joel on Software*, pages 197–202. Springer, 2004
- H. Psaier and S. Dustdar. A survey on self-healing systems: Approaches and systems.
  *Computing (Vienna/New York)*, 91(1):43–73, 2011
- B. Fitzgerald. Software crisis 2.0.
  *Computer*, 45(4):89–91, 2012
- C. Prehofer and L. Chiarabini. From Internet of things mashups to model-based development.
  *Proceedings - International Computer Software and Applications Conference*, 3:499–504, 2015
- R. Buyya and A. V. Dastjerdi. *Internet of Things: Principles and Paradigms*.
  Elsevier, 2016
- M. Weyrich and C. Ebert. Reference architectures for the internet of things.
  *IEEE Software*, 33(1):112–116, 2016
- S. Smith. *The Internet of Risky Things*.
  O'Reilly Media, Inc., 2017

# References II


- B. Morin, N. Harrand, and F. Fleurey. Model-Based Software Engineering to Tame the IoT Jungle. *IEEE Software*, 34(1):30–36, 2017
- A. Taivalsaari and T. Mikkonen. A Roadmap to the Programmable World: Software Challenges in the IoT Era. *IEEE Software*, 34(1):72–80, 2017
- B. Cheng, E. Kovacs, A. Kitazawa, and et al. Fogflow: Orchestrating iot services over cloud and edges. *NEC Technical Journal*, 13:48–53, 11 2018
- A. Seitz, F. Thiele, and B. Bruegge. Fogxy: An Architectural Pattern for Fog Computing. In *Proceedings of the 23rd European Conference on Pattern Languages of Programs*, volume 1, page 33. ACM, ACM, 2018
- Microsoft. Iot signals – summary of research learnings. Technical report, Microsoft, 2019
- T. Ammari, J. Kaye, J. Y. Tsai, and F. Bentley. Music, search, and iot: How people (really) use voice assistants. *ACM Transactions in Computer-Human Interaction*, 26(3), Apr. 2019
- M. Kleppmann, A. Wiggins, P. Hardenberg, and M. McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, Onward! 2019, page 154–178, New York, NY, USA, 2019. Association for Computing Machinery
- F. Ihirwe, D. Di Ruscio, and et al. Low-code engineering for internet of things: a state of research. In *23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, pages 1–8, 2020
- M. Langheinrich. Long live the iot. *IEEE Pervasive Computing*, 19(2):4–7, 2020
- A. Makhshari and A. Mesbah. Iot bugs and development challenges. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 460–472, 2021

# Increasing the Dependability of Internet-of-Things Systems in the context of End-User Development Environments

João Pedro Dias

jpmdias@fe.up.pt

Supervision by:
Hugo Sereno Ferreira, PhD
João Pascoal Faria, PhD

In partial fulfillment of requirements for the degree of *Doctor of Philosophy in Informatics Engineering* by the Doctoral Program in Informatics Engineering (ProDEI)

April 1, 2022 — Porto, Portugal

U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO